

Universidad Autónoma Metropolitana

Licenciatura en tecnologías y sistemas de la información.

Seminario de Tecnologías II

Tarea 5

Instancing y Vertex Animation Textures

Diego Barrera Hernández

Brandon Daniel Martínez Alcántara

Ciudad de México, 2025

Índice

1. Objetivo	2
2. Escena implementada: Cardumen de peces	2
3. Parte D — Instancing con InstancedMesh	3
3.1. El concepto de Instancing	3
3.2. Inicialización y pre-asignación	3
3.3. Bucle de actualización de instancias	4
3.4. Simulación de natación	4
3.5. Control de cantidad	5
4. Parte E — Bone VAT (Vertex Animation Texture)	5
4.1. El concepto de VAT	5
4.2. Horneado: <code>bakeBoneVAT</code>	5
4.3. Offset de tiempo por instancia	6
4.4. Vertex shader — lectura de la textura VAT	6
4.5. Fragment shader — iluminación Blinn-Phong con textura	7
4.6. Trabajo de CPU por frame	7
5. Estructura del proyecto	8
6. Comparativa de los dos enfoques	8
7. Despliegue	9
8. Conclusiones	9

1. Objetivo

El objetivo de esta tarea es comparar dos enfoques para renderizar grandes cantidades de agentes animados en tiempo real: **Instancing con actualización de matrices en CPU** (Parte D) y **Vertex Animation Textures horneadas** (Parte E). Ambas partes usan `THREE.InstancedMesh` para emitir un único draw call independientemente del número de instancias, pero difieren radicalmente en dónde vive el costo de animación.

El proyecto se divide en dos partes:

- **Parte D (50 pts):** `InstancedMesh` con movimiento simulado — las matrices de cada instancia se recalculan en CPU cada frame.
- **Parte E (50 pts):** `InstancedMesh` + Bone VAT — la animación de esqueleto se hornea en una `DataTexture` y el vertex shader la reproduce en GPU sin ningún trabajo en CPU por frame.

La demo desplegada está disponible en:

<https://tareas-seminario.vercel.app/>

2. Escena implementada: Cardumen de peces

Ambas partes trabajan sobre la misma escena: un cardumen de peces payaso (*clown fish*) renderizados en un entorno subacuático oscuro con niebla exponencial. El modelo `clown_fish_low_poly_animated.glb` es un `SkinnedMesh` con animación de esqueleto (`swim`) cargado mediante `GLTFLoader`.

La escena incluye:

- Fondo oscuro (`#071521`) con niebla exponencial ($density = 0.014$).
- Luz hemisférica (cielo azulado / fondo oscuro) + luz direccional cálida.
- Suelo circular con `MeshStandardMaterial` rugoso.
- Cámara con `OrbitControls` (`damping 0.08`) posicionada en $(0, 16, 48)$.
- Límites de movimiento ± 30 en X y Z , $[0.6, 11]$ en Y .

El modelo se normaliza al cargar: se centra en el origen, se escala uniformemente para que su mayor dimensión mida 2.6 unidades y se rota 180° en Y para que los peces apunten en la dirección de avance.

3. Parte D — Instancing con InstancedMesh

3.1. El concepto de Instancing

Instancing permite renderizar N copias de la misma geometría con un único draw call. En lugar de llamar N veces a `drawElements`, la GPU recibe la geometría una vez y un buffer de matrices que describe la posición, rotación y escala de cada copia. Esto elimina el overhead de CPU de N draw calls y es especialmente eficiente cuando las instancias comparten la misma malla y material.

En Three.js, `THREE.InstancedMesh` encapsula este mecanismo: acepta una `BufferGeometry`, un `Material` y el número máximo de instancias, y expone `setMatrixAt(i, matrix)` para actualizar la transformación de cada instancia.

3.2. Inicialización y pre-asignación

El sistema pre-asigna **2 000 instancias** al crear el `InstancedMesh`, lo que permite cambiar la cantidad activa en tiempo de ejecución sin reasignar buffers de GPU. El estado de simulación de cada instancia vive en `Float32Array` paralelos:

```
const MAX_INSTANCES = 2000;
const posX          = new Float32Array(MAX_INSTANCES);
const posY          = new Float32Array(MAX_INSTANCES);
const posZ          = new Float32Array(MAX_INSTANCES);
const heading       = new Float32Array(MAX_INSTANCES);
const speed         = new Float32Array(MAX_INSTANCES);
const phase         = new Float32Array(MAX_INSTANCES);
const scaleArr      = new Float32Array(MAX_INSTANCES);
const turnRate      = new Float32Array(MAX_INSTANCES);
```

Los arreglos de tipo (`Float32Array`) se eligen deliberadamente en lugar de arrays JavaScript normales: son datos contiguos en memoria y permiten iteraciones más rápidas en el bucle de actualización al evitar la indiferencia de tipo del motor JS.

La carga del modelo elimina los atributos `skinIndex` y `skinWeight` (innecesarios en esta parte) y crea un único `InstancedMesh`:

```
geometry.deleteAttribute('skinIndex');
geometry.deleteAttribute('skinWeight');
instancedMesh = new THREE.InstancedMesh(geometry, material,
    MAX_INSTANCES);
instancedMesh.instanceMatrix.setUsage(THREE.DynamicDrawUsage);
instancedMesh.count = activeCount; // comienza en 500
```

`DynamicDrawUsage` indica a la GPU que el buffer de matrices se actualizará frecuentemente, lo que puede mejorar el rendimiento en algunas implementaciones WebGL.

3.3. Bucle de actualización de instancias

Cada frame, la función `updateInstances(dt, elapsed)` itera sobre las `activeCount` instancias y calcula la nueva posición y orientación:

```
for (let i = 0; i < count; i++) {
  // Giro suave: componente determinista + oscilacion senoidal
  heading[i] += turnRate[i] * dt * 0.2
               + Math.sin(elapsed * 0.5 + phase[i]) * dt * 0.3;

  // Avance en la direccion actual
  posX[i] += Math.cos(heading[i]) * speed[i] * dt;
  posZ[i] += Math.sin(heading[i]) * speed[i] * dt;
  posY[i] += Math.sin(elapsed * 1.3 + phase[i]) * dt * 0.6;

  // Wrapping toroidal en los limites del volumen
  if (posX[i] > BOUNDS.x) posX[i] = -BOUNDS.x;
  if (posX[i] < -BOUNDS.x) posX[i] = BOUNDS.x;
  // ... idem Z, Y con clamp

  dummy.position.set(posX[i], posY[i], posZ[i]);
  dummy.rotation.set(Math.sin(elapsed*4 + phase[i])*0.08,
                     heading[i], 0);
  dummy.scale.setScalar(scaleArr[i]);
  dummy.updateMatrix();
  instancedMesh.setMatrixAt(i, dummy.matrix);
}
instancedMesh.instanceMatrix.needsUpdate = true;
```

Al final del bucle, `instanceMatrix.needsUpdate = true` sube el buffer de matrices completo a la GPU en el siguiente frame. Esto es el costo dominante de la Parte D: $O(N)$ operaciones en CPU por frame y una transferencia de $N \times 64$ bytes (cada `mat4`) al bus de la GPU.

3.4. Simulación de natación

El movimiento visible de los peces combina tres componentes, todas calculadas en CPU:

Componente	Mecanismo
Avance horizontal	$\cos(\text{heading}) \times \text{speed} \times dt$
Oscilación vertical	$\sin(\text{elapsed} \times 1.3 + \text{phase}_i) \times 0.6 \times dt$
Bamboleo de cabeza	Rotación en X : $\sin(\text{elapsed} \times 4 + \text{phase}_i) \times 0.08$
Giro	$\text{turnRate}_i + \text{perturbación senoidal lenta}$

La propiedad `phasei` inicializada aleatoriamente garantiza que cada pez esté en un punto distinto de su ciclo de movimiento desde el primer frame.

3.5. Control de cantidad

Un slider en el panel lateral cambia `instancedMesh.count` sin reconstruir nada: Three.js simplemente ignora las instancias con índice mayor al `count` activo. Esto permite observar en tiempo real cómo el FPS cae a medida que aumenta el número de instancias activas, ilustrando la complejidad $O(N)$ del enfoque.

4. Parte E — Bone VAT (Vertex Animation Texture)

4.1. El concepto de VAT

Una *Vertex Animation Texture* (VAT) o, en este caso, *Bone VAT*, consiste en hornear los resultados de una animación de esqueleto a una textura y reproducirla en el vertex shader. En lugar de ejecutar el `AnimationMixer` de Three.js en CPU cada frame (que calcula e interpola la pose de todos los huesos), el shader simplemente **muestrea la textura** para obtener la matriz de cada hueso en el instante deseado.

El resultado es que el costo de animación pasa de $O(N \times B)$ en CPU (N instancias, B huesos) a un costo fijo de GPU que depende solo de la complejidad del shader, no del número de instancias.

4.2. Horneado: bakeBoneVAT

La función `bakeBoneVAT(gltf, clipName)` recorre el clip de animación a **24 fps** y almacena la pose de cada hueso en cada frame:

1. Se localiza el `SkinnedMesh` y su `Skeleton` dentro de la escena GLTF.
2. Se crea un `AnimationMixer` efímero y se hace avanzar al clip frame a frame con `mixer.setTime(t) + skeleton.update()`.
3. En cada frame f , `skeleton.boneMatrices` contiene el arreglo de matrices 4×4 de todos los huesos ya multiplicadas por la `bindMatrixInverse` (listas para el skinning). Estas 16 floats por hueso se copian a un `Float32Array` de tamaño `numFrames × numBones × 16`.
4. Los datos se suben como `DataTexture` de tipo `FloatType`, con `NearestFilter` (para evitar interpolación no deseada entre huesos) y sin mipmaps.

```
const BAKE_FPS = 24;
const numFrames = Math.round(clip.duration * BAKE_FPS);
const data = new Float32Array(numFrames * numBones * 16);

for (let f = 0; f < numFrames; f++) {
  mixer.setTime((f / (numFrames - 1)) * clip.duration);
  gltf.scene.updateMatrixWorld(true);
}
```

```

    skeleton.update();
    data.set(skeleton.boneMatrices, f * numBones * 16);
}

const texture = new THREE.DataTexture(
    data, numBones * 4, numFrames,
    THREE.RGBAFormat, THREE.FloatType
);
texture.magFilter = THREE.NearestFilter;
texture.needsUpdate = true;

```

La textura tiene **ancho** = $4 \times \text{numBones}$ texeles (cada mat4 ocupa 4 texeles RGBA de 4 floats cada uno) y **alto** = **numFrames** filas. Una vez creada, el mixer se descarta: ya no se necesita en ningún frame posterior.

4.3. Offset de tiempo por instancia

Para que las 500 instancias nadan de forma desincronizada sin ningún costo adicional por frame, se añade un atributo `timeOffset` como `InstancedBufferAttribute`:

```

const timeOffsets = new Float32Array(INSTANCE_COUNT);
for (let i = 0; i < INSTANCE_COUNT; i++) {
    timeOffsets[i] = Math.random() * baked.duration;
}
geometry.setAttribute('timeOffset',
    new THREE.InstancedBufferAttribute(timeOffsets, 1));

```

Este buffer se sube a la GPU una única vez al cargar. El vertex shader usa `timeOffset` (por instancia) más el uniforme `time` (global) para calcular el frame de animación de cada instancia independientemente:

```

float localTime = mod(time + timeOffset, duration);
float framef    = (localTime / duration) * numFrames;
float f0        = floor(framef);
float f1        = mod(f0 + 1.0, numFrames);
float frac      = framef - f0;

```

4.4. Vertex shader — lectura de la textura VAT

La función auxiliar `getBoneMatrix` reconstruye una mat4 a partir de cuatro texeles consecutivos de la fila correspondiente al frame:

```

mat4 getBoneMatrix(float boneIndex, float frame) {
    float texWidth = numBones * 4.0;
    float u0 = (boneIndex * 4.0 + 0.5) / texWidth;
    float du = 1.0 / texWidth;
    float v = (frame + 0.5) / numFrames;
    vec4 c0 = texture2D(boneTexture, vec2(u0, v));
    vec4 c1 = texture2D(boneTexture, vec2(u0 + du, v));
    vec4 c2 = texture2D(boneTexture, vec2(u0 + 2.0*du, v));
    vec4 c3 = texture2D(boneTexture, vec2(u0 + 3.0*du, v));
}

```

```
    return mat4(c0, c1, c2, c3);
}
```

Con las matrices de los frames f_0 y f_1 se calcula el skinning en ambos y se interpola linealmente con mix para conseguir una animación suave entre fotogramas horneados:

```
vec4 skinVertex = bindMatrix * vec4(position, 1.0);

mat4 bx0 = getBoneMatrix(skinIndex.x, f0);
// ... (by0, bz0, bw0 idem para los otros 3 huesos)
mat4 bx1 = getBoneMatrix(skinIndex.x, f1);
// ... (by1, bz1, bw1)

vec4 pos0 = bx0*skinVertex*skinWeight.x + by0*skinVertex*skinWeight.y
           + bz0*skinVertex*skinWeight.z + bw0*skinVertex*skinWeight.w;
vec4 pos1 = bx1*skinVertex*skinWeight.x + by1*skinVertex*skinWeight.y
           + bz1*skinVertex*skinWeight.z + bw1*skinVertex*skinWeight.w;

vec4 skinnedPos = mix(pos0, pos1, frac);
vec3 localPos   = (bindMatrixInverse * skinnedPos).xyz;
```

La posición final en clip space combina la transformación de instancia (instanceMatrix, fijada una sola vez al cargar) con la posición de modelo:

```
vec4 worldPos = modelMatrix * instanceMatrix * vec4(localPos, 1.0);
gl_Position   = projectionMatrix * viewMatrix * worldPos;
```

4.5. Fragment shader — iluminación Blinn-Phong con textura

El fragment shader aplica la textura de color del modelo con iluminación Blinn-Phong usando el half-vector $\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$:

```
vec3 N = normalize(vNormal);
vec3 L = normalize(-lightDir);
vec3 V = normalize(viewPos - vWorldPos);
vec3 H = normalize(L + V);

vec4 base = texture2D(map, vUv);
float diff = max(dot(N, L), 0.0);
float spec = pow(max(dot(N, H), 0.0), 32.0);

vec3 color = base.rgb * 0.35 // termino ambiental
           + base.rgb * diff * lightColor // difuso
           + vec3(0.18) * spec;          // especular

gl_FragColor = vec4(color, base.a);
```

4.6. Trabajo de CPU por frame

Una vez terminada la carga, el único trabajo de CPU por frame es:

```
if (material) material.uniforms.time.value = elapsed;
```


Una asignación de un flotante. Las matrices de instancia **nunca se tocan** tras la inicialización: los peces no se mueven por el espacio en la Parte E (cada uno permanece en su posición fija), pero su ciclo de natación es visible porque el vertex shader los deforma desde la textura VAT.

5. Estructura del proyecto

```
Tarea_3/
+-- partD/
|   +-- index.html      (interfaz: slider de cantidad, stats)
|   +-- main.js         (punto de entrada)
+-- partE/
|   +-- index.html      (interfaz: stats del VAT)
|   +-- main.js         (punto de entrada)
+-- src/
    +-- shared/
    |   +-- FpsMeter.js  (contador de frames en pantalla)
    +-- instancing/
    |   +-- FishSwarm.js (Parte D: InstancedMesh + sim CPU)
    +-- vat/
        +-- BoneVAT.js   (horneado de clip a DataTexture)
        +-- FishSwarmVAT.js (Parte E: InstancedMesh + ShaderMaterial)
        +-- boneVatShader.js (vertex y fragment GLSL)
```

6. Comparativa de los dos enfoques

Aspecto	Parte D (Instancing)	Parte E (Bone VAT)
Draw calls por frame	1	1
Costo de animación	$O(N)$ en CPU	$O(1)$ en CPU
Trabajo CPU por frame	N matrices actualizadas	1 float (time)
Transferencia GPU/frame	$N \times 64$ bytes	4 bytes (uniforme)
Desincronización	phase_i + sim	timeOffset_i
Skinning	Ninguno (forma estática)	GPU, vertex shader
Variación de cantidad	Slider 100–2000	Fijo en 500
Modelo usado	Solo la malla (sin rig)	Malla + rig + clip

La diferencia clave es que en la Parte D el movimiento de cada pez se simula matemáticamente en CPU sin skinning real, mientras que la Parte E reproduce la animación de esqueleto original del modelo sin ningún trabajo de CPU por frame.

7. Despliegue

El proyecto se construye con **Vite** (`npm run build`) generando un sitio estático en `dist/`. Está desplegado en **Vercel** con configuración automática detectada por el preset de Vite:

- **Build command:** `npm run build`
- **Output directory:** `dist`
- **Root directory:** `Tarea_3`

URL pública: <https://tareasseminario.vercel.app/>

Las Partes D y E usan `THREE.WebGLRenderer` estándar, por lo que funcionan en cualquier navegador moderno con soporte WebGL2 (Chrome 56+, Firefox 51+, Safari 15+).

8. Conclusiones

Implementar el mismo cardumen de peces con dos estrategias de animación distintas permite cuantificar con precisión el trade-off entre flexibilidad y rendimiento:

- **Instancing puro (Parte D)** es sencillo de implementar: basta con mantener arreglos de estado por instancia y actualizar las matrices cada frame. Su limitación es que el costo escala linealmente con el número de instancias activas, lo que pone un techo práctico de rendimiento visible en tiempo real con el slider.
- **Bone VAT (Parte E)** requiere un paso de preprocesamiento (el horneado) y un shader personalizado más complejo, pero la recompensa es que el costo de animación en CPU por frame es constante e independiente de cuántas instancias haya. Esto hace al enfoque VAT ideal para efectos de masas donde se necesitan cientos o miles de personajes animados simultáneamente.
- Ambas partes mantienen **un único draw call** gracias a `InstancedMesh`, confirmando que el cuello de botella en la Parte D está en la CPU (actualización de matrices), no en la GPU.
- El atributo `timeOffset` de la Parte E demuestra que la desincronización de instancias es trivial con VAT: el offset se sube una vez y la GPU se encarga del resto sin ningún dato adicional por frame.